

Technical Report

Asynchronous MPC Integration & PID Trajectory Tracking

Date: May 2026

1. Asynchronous MPC vs. Synchronous MPC in ROS2

Model Predictive Control (MPC) is highly effective for optimal trajectory planning, but solving non-linear optimization problems (via Solvers like CasADi/IPOPT) is computationally expensive. The standard synchronous `mpc_controller.py` encounters significant limitations in real-time robotic systems, which the `async_mpc_controller_node.py` is designed to resolve.

1.1 The Synchronous Bottleneck

In a synchronous implementation, the ROS2 node executes the MPC solver inside the main timer callback. When the solver takes 50–150ms to converge:

- **Executor Blocking:** The single-threaded executor cannot process incoming `/odom` (Odometry) and laser scan callbacks.
- **Stale State Estimation:** By the time the solver outputs a velocity command, the robot has already moved. The computed velocities are based on old states, causing erratic oscillations.
- **Infeasibility Loops:** The delayed state feedback leads to diverging error states, causing the solver to repeatedly return an `AMPC_INFEASIBLE` status, paralyzing the robot.

1.2 The Asynchronous Multi-Threaded Architecture

The `async_mpc_controller_node.py` solves this by decoupling the control loop from the optimization loop using Python's `threading` module:

- **State Thread (ROS2 Executor):** Runs continuously at high frequency, updating odometry and obstacle data without interruption.
- **Solver Thread (Background):** Runs a continuous `while rclpy.ok()` loop. It securely locks the latest state, solves the CasADi non-linear programming problem, and stores the optimal control effort `(v, omega)`.
- **Publisher Timer (Foreground):** A lightweight ROS2 timer executes at a strict 30Hz, reading the most recently available `(v, omega)` from the solver thread and publishing to `/cmd_vel`.

Crucial Advantage: Emergency Overrides

Because the publisher loop is decoupled, it can react instantly to environmental changes. As implemented in the codebase, if the robot's state thread detects an obstacle within 0.35m, the publisher bypasses the MPC entirely and instantly issues an **Emergency Reverse** command ($v = -0.50, \omega = 2.0$). This guarantees collision avoidance without waiting for the slow optimization solver to converge.

2. PID Controller Implementation for Lissajous Tracking

The standalone `pid_lissajous_sim.py` module verifies the base line-tracking capabilities of a differential-drive robot following a continuous Figure-8 (Lissajous) trajectory using Proportional-Integral-Derivative (PID) control.

2.1 Mathematical Formulation of the Reference

The infinity-shaped Lissajous curve is parameterized by time t :

$$x_r(t) = A \sin(\omega t)$$

$$y_r(t) = B \sin(2\omega t)$$

To align the robot tangent to the path, the reference heading $\theta_r(t)$ is determined via the analytical derivatives of the curve:

$$\dot{x}_r(t) = A\omega \cos(\omega t)$$

$$\dot{y}_r(t) = 2B\omega \cos(2\omega t)$$

$$\theta_r(t) = \text{atan2}(\dot{y}_r(t), \dot{x}_r(t))$$

2.2 Error Computation

At any instant k , the robot is at state $[x, y, \theta]$. The control errors are:

$$e_{dist} = \sqrt{(x_r - x)^2 + (y_r - y)^2}$$

The desired heading angle points directly at the reference point:

$\theta_{des} = \text{atan2}(y_r - y, x_r - x)$. The angular error is wrapped to $[-\pi, \pi]$:

$$e_{head} = (\theta_{des} - \theta) \pmod{2\pi}$$

Note: If $e_{dist} < 0.05m$, θ_{des} switches to $\theta_r(t)$ to prevent rapid spin oscillations when intersecting the exact reference point.

2.3 Dual PID Control Laws

The system utilizes two independent PID loops. The linear velocity v incorporates a directional projection factor $\cos(e_{head})$ to prevent the robot from driving forward when it is facing away from the target:

$$v_{cmd} = \left(K_{p,v} e_{dist} + K_{i,v} \int e_{dist} dt + K_{d,v} \frac{de_{dist}}{dt} \right) \cdot \cos(e_{head})$$

$$\omega_{cmd} = K_{p,\omega} e_{head} + K_{i,\omega} \int e_{head} dt + K_{d,\omega} \frac{de_{head}}{dt}$$

2.4 Algorithmic Implementation Details

- **Anti-Windup Clamp:** The discrete integral term $I_{err} += e \cdot \Delta t$ is strictly clipped to $[\text{out}_{min}/K_i, \text{out}_{max}/K_i]$. This prevents the integral from accumulating excessively during large turns, avoiding persistent overshoots.
- **Kinematic Integration:** The computed velocities are clamped to physical limits ($|v| \leq 2.0$ m/s, $|\omega| \leq 3.0$ rad/s) and passed through the unicycle kinematic model:

$$x_{k+1} = x_k + v_{cmd} \cos(\theta_k) \Delta t$$

$$y_{k+1} = y_k + v_{cmd} \sin(\theta_k) \Delta t$$

$$\theta_{k+1} = \theta_k + \omega_{cmd} \Delta t$$

- **Visualization:** The implementation leverages `matplotlib.animation` running at 50Hz, actively logging position, reference paths, and real-time control efforts to empirically validate the system's stability.